

TRIANGLE HEALTHY KITCHEN

Software Handover

Complete operating, customer, technical, deployment, and external services guide

Documentation set 1.0 | Reviewed 3 October 2026

Prepared for the client and the next team responsible for operating and maintaining the platform.

Contents

Triangle Healthy Kitchen Software Handover	5
What this package covers	5
Read these first	5
Current delivery snapshot	5
Access and data handling	6
Commercial paperwork is separate	6
Handover completion record	6
Customer Guide	7
Using Triangle Healthy Kitchen	7
Start or renew a plan	7
Payment choices	7
Weekly meals and deadlines	7
Friday meals and delivery	8
Delivery details	8
Language and mobile health data	8
Account help	8
Operations Guide	9
Where each role works	9
Sign-in and role access	9
Role areas in the web portal	9
Daily operating rhythm	9
CEO/Admin	9
Kitchen	10
Transport	10
Rider	10
Escalation rules	10
Kitchen Production Guide	11
The kitchen's work is built around totals first, customer-specific packing second	11
Monthly menu release	11
Customer choice and cutoff	11
Daily production board	11
Weekly bulk purchasing	12

Packing and portioning	12
Portions and recipes to configure before go-live	12
Payments and Subscriptions	13
Source of truth	13
Tap online payments	13
Intended flow	13
If the return page loops or says pending	13
Cash collection	13
Demo records	14
Payment provider and notification setup	14
Technical Handover	15
System overview	15
Main data areas	15
Access model	16
Data flow	16
Configuration names - values are not included here	16
Web/mobile client build environment	16
Supabase server-side functions and vault	16
Native mobile configuration	16
Integration inventory and state	17
Critical repository cautions	17
Useful source entry points	18
Deployment and Maintenance	19
Account ownership	19
Web development and release	19
Mobile build and release	19
Supabase/database/functions release	19
Backups and recovery	20
Routine maintenance calendar	20
Support and maintenance terms	20
Launch Checklist and Known Limits	21
A. Ownership and access	21
B. Customer signup and account	21

C. Plan, menu, and kitchen	21
D. Payments	22
E. Email, mobile, and integrations	22
F. Release and operations acceptance	22
Known limitations and handover conditions found in repository review	23
Sign-off	23
External Services Runbook	24
Service inventory	24
1. Supabase	24
Ownership and initial access	24
Auth and email setup	25
Database, storage, functions, and scheduled jobs	25
If something fails	25
2. Vercel	25
3. Brevo	26
Supabase Auth transactional SMTP	26
THK Edge Function emails	26
4. Tap Payments	26
5. Firebase Cloud Messaging (Android)	27
6. Apple Health and Android Health Connect	27
iPhone / Apple Health	27
Android / Health Connect	27
7. Location and maps	28
8. WhatsApp, social links, and remote images	28
Provider incident checklist	28

Triangle Healthy Kitchen Software Handover

Documentation set: 1.0 **Reviewed:** 3 October 2026 **Product:** Triangle Healthy Kitchen customer platform, operations portal, and mobile apps

What this package covers

This handover set explains how customers use the service, how each operations team works, how menus and production flow through the system, how payments are reconciled, and what the client or next developer must configure before a full launch. It is written against the current `thk-web/` and `thk-mobile/` source trees. It does not include source code, customer exports, database credentials, payment keys, Firebase configuration files, signing keys, or signed contracts.

Read these first

- 1. **Customer Guide** - account, package, meal selection, delivery details, payment, and mobile health connections.
- 2. **Operations Guide** - the web dashboards for CEO/Admin, Kitchen, and Transport, plus the Rider mobile workflow.
- 3. **Kitchen Production Guide** - daily production, weekly bulk purchasing, portions, safety notes, and packing.
- 4. **Payments and Subscriptions** - Tap and cash processing, review steps, plan states, menu schedule, and customer retention.
- 5. **Technical Handover** - architecture, roles, data flows, integrations, configuration names, security, and migration cautions.
- 6. **Deployment and Maintenance** - web releases, database/functions, mobile builds, backups, recovery, and ongoing responsibilities.
- 7. **Launch Checklist and Known Limits** - acceptance checks and items that need production-owner confirmation.
- 8. **External Services Runbook** - setup, verification, ownership, and failure handling for Supabase, Vercel, Brevo, Tap, Firebase, health platforms, maps/geocoding, WhatsApp, and external assets.

Current delivery snapshot

Area	Current state described by source/configuration	Before calling it production-ready
Web	Live Vercel site: trianglehealthykitchen.vercel.app	Run the client's final acceptance checks on the live domain.
Customer mobile app	Capacitor iOS and Android projects are in <code>thk-mobile/</code> ; app features use the shared customer product.	Build, sign, and distribute through the client's Apple/Google accounts. Store approval is external and not included in this code handover.
Operations mobile access	CEO, Admin, Kitchen, and Transport are directed to the web operations portal. Riders use the mobile app.	Confirm role assignments and portal access for named staff.
Menu and kitchen	Monthly menu creation, weekly meal selections, production views, ingredient totals, and packing views are present in source/migrations.	Confirm the production database has the matching migrations, menu data, recipe quantities, scheduled jobs, and the client-approved portion ranges. Recipe quantities currently require kitchen validation.
Payments	Tap checkout/callback and manual reconciliation are present; cash collection requests and verification are present.	Confirm live provider credentials, webhook delivery, amount/currency, and one real end-to-end transaction. Tap callback was not independently certified by this documentation review.
Sadad	No Sadad checkout implementation was found in the reviewed product source.	Do not advertise Sadad as an available in-app payment method unless it is separately implemented and tested.

Area	Current state described by source/configuration	Before calling it production-ready
Emails/reminders	Brevo-backed Edge Function code and a scheduled reminder job exist in the repository copies.	Confirm the deployed function source, Brevo secrets/sender verification, and Supabase cron job on the production project.
Android push	Android push registration is conditional on a Firebase Android config file. A local Firebase config file is not included in this documentation bundle.	Set up under the client-owned Firebase project, secure the file, rebuild, and test on a real device.
iOS distribution	The iOS project is present.	Client Apple Developer membership, HealthKit entitlement/signing, signed archive, privacy disclosures, and App Store review are required for store release.

"Present in source" does not prove that a feature is deployed, configured, or accepted in the production environment. The launch checklist identifies those checks explicitly.

Access and data handling

- The client should own or control Vercel, Supabase, Tap, Brevo, Apple Developer, Google Play, and Firebase accounts.
- Transfer access through each provider's invitation/role system. Never put passwords, service-role keys, Tap secrets, webhook secrets, signing keys, or private Firebase files in this bundle, email, Git, or a support ticket.
- The browser/mobile app's Supabase anon key is a public client key; it is not a substitute for RLS and must never be confused with the service-role key.
- Do not send real client health, allergy, address, or payment information in sample screenshots or support messages.

Commercial paperwork is separate

The supplied fee, license, and referral agreements are working drafts, not product manuals. Use one agreed master agreement and make its fee, license scope, support obligations, referrals, and app-store scope consistent with the EULA before anyone signs. This documentation set does not change or replace those agreements.

Handover completion record

Complete this with the client at handover:

- Production URL and Vercel project owner: _____
- Supabase project owner and project reference: _____
- Production database migration review completed by/date: _____
- Tap live configuration and test transaction verified by/date: _____
- Brevo sender and reminder schedule verified by/date: _____
- Firebase Android push configured/tested by/date: _____
- Apple signing/store release owner/date: _____
- Staff roles and initial access verified by/date: _____
- Backup/restore rehearsal owner/date: _____
- Client acceptance owner/date: _____

Customer Guide

Using Triangle Healthy Kitchen

Customers can use the live website or the iOS/Android app to review plans, create an account, choose meals, provide delivery details, and manage an active subscription. The mobile app is for customer and rider tasks; company operations dashboards are on the web.

Start or renew a plan

1. Open the website or app and choose **Start your plan**.
2. Complete the profile and health-goal questions. Enter realistic age, weight, and height values. A BMI report is optional; it is shared for the nutrition team's review and is not a substitute for medical advice.
3. Choose a package. Package availability, name, price, meal periods, and duration are loaded from the active package records. Confirm the current values shown in the app before paying; the code's fallback examples may not match later admin-edited package records.
4. Choose the available meals for the upcoming service week. The kitchen's choice is preselected where one is available. Keep that selection if you are happy with it; change only the meals you want to change.
5. Provide allergies and kitchen preparation notes. These are shared with Kitchen for food safety and preparation. Describe ingredients to avoid clearly; contact the business directly for urgent allergy questions.
6. Enter the place that should receive the food: home, office, gym, or another location. Use **Locate** if you are physically there, or enter the address manually if you are elsewhere. Review the building, street, area, and map pin. Confirm the suggested delivery zone with the business if the address is not recognized.
7. Create/sign in to your account, accept the displayed terms, and choose the payment method offered.
8. Keep the confirmation/reference. If checkout returns without showing an active plan, sign in and check the plan status before trying to pay again. A pending payment should be reconciled by Admin/CEO first to avoid duplicate charges.

Payment choices

- **Tap:** Continue to Tap's secure checkout. The plan activates after a verified capture. If the return page says pending, do not assume the payment failed; contact the business with the transaction reference so staff can check Tap and the operations payment log.
- **Cash collection:** The request remains pending until Admin/CEO confirms that cash was physically received. The confirmation message says the team will reach out. Do not treat an unverified cash request as an active plan.
- Sadad is not documented as an available integrated checkout in the current source. Use only the payment options actually displayed.

Weekly meals and deadlines

- The service week starts Saturday. The menu is intended to open on Saturday for the upcoming week.
- Change meal choices before **Thursday 11:59 PM Qatar time**. The app should preselect Kitchen's choice when available, so customers who accept it do not need to reselect it.
- After the deadline, the scheduled kitchen-choice process fills any missing meal choices. Confirm that the client's production environment has the scheduled database job enabled; see the Launch Checklist.
- A reminder email is intended to go out about 24 hours before the deadline to customers with incomplete meal choices. Email delivery depends on the production Brevo configuration and scheduled job.
- If there is no new release for the upcoming week, the system has a database function to roll the latest prior menu forward. Operations should still check the menu before the week opens and confirm the fallback job is active.

Friday meals and delivery

The optional Friday add-on is **QAR 199 per monthly subscription** and includes Friday meal selections and the Friday delivery. It is not an extra charge per Friday. It is not offered with the one-day or six-day trial in the current flow. Confirm the total displayed before payment.

Delivery details

Each customer receives **one delivery stop per service day**, with all meals for that day together. The delivery address and zone help Transport group stops and assign riders. If you move between home, office, and gym, update the active delivery address/details through the account flow as supported and tell the business about a change that affects an imminent route.

Language and mobile health data

- Use the language control to switch between English and Arabic. If wording appears untranslated or confusing, capture the screen and report the page and language; do not assume a partial translation changes plan or payment rules.
- Health sync is optional and customer-initiated. iPhone uses Apple Health/HealthKit; Android uses Health Connect. The customer must grant read access on their device. Samsung Health data must be shared with Health Connect first. If unavailable, continue using the product without syncing. The app requests steps, distance, calories, and weight permissions in the mobile implementation; actual data availability depends on device, OS, permissions, and signing/configuration.

Account help

Use the password-reset link sent to the email on the account and open it on the live domain. A reset page that reports a missing/expired auth session needs a fresh recovery email and valid session; do not repeat the password update from an old link. For a plan that is pending after payment, ask Admin/CEO to reconcile the transaction before starting another checkout.

The account has a **Request account deletion** flow. The app tells the customer the request is processed within 30 days. Separately, the database contains a scheduled cleanup for non-owner, non-staff, non-rider customer accounts whose subscription period ended more than 30 days earlier. Confirm production scheduling and the published privacy policy before promising retention/deletion dates beyond what the screen says.

Operations Guide

Where each role works

Role	Main workspace	Intended work
CEO	Web operations portal	Oversight and all operating areas.
Admin	Web operations portal	Customers, subscriptions, menus, payments, reports, team/settings, and daily administration.
Kitchen	Web operations portal	Menu catalogue/release, daily production, ingredient purchasing totals, allergies/preparation notes, and packing.
Transport	Web operations portal	Delivery addresses/zones, rider assignment, routes, delivery status, and rider fleet.
Rider	Mobile app	Assigned stops, delivery execution, and availability.
Customer	Website or mobile app	Subscription, meals, delivery details, health/rewards, and account settings.

The mobile app routes CEO/Admin/Kitchen/Transport to the web portal. Do not look for a Kitchen or Transport dashboard inside the mobile app. Riders are the operations role intended to work in mobile.

Sign-in and role access

1. Use the staff member's own approved sign-in account. Do not share a personal owner account among staff.
2. The web role is read from the authenticated staff-role record; the interface is not the only security boundary. Supabase policies/views/functions must also be present and current.
3. After sign-in, confirm the workspace label and tabs match the staff member's role. If access is wrong, stop and ask the system administrator to correct the role assignment; do not work around it by sharing another role's credentials.
4. Sign out on shared devices. Remove access promptly when a staff member leaves.

Role areas in the web portal

- **CEO/Admin:** Dashboard, Customers, Subscriptions, Meal Plans, Kitchen, Packing, Delivery, Drivers, Payments, Reminders, Reports, Consultations, Team, Settings. Actual visible tabs are role-filtered.
- **Kitchen:** Dashboard, Meal Plans, Kitchen, Packing, Reports, and the current source also shows Consultations in the tab list. The product owner previously intended Kitchen not to handle customer consultations; confirm whether this remaining visible tab should be removed before final acceptance.
- **Transport:** Dashboard, Delivery, Drivers.

The current source reads only already-approved rider records for the fleet screen. It does not show a clear pending-rider approval control in the operations dashboard code reviewed for this handover. Treat rider application approval as an outstanding admin process until a supported approval screen/workflow is confirmed.

Daily operating rhythm

CEO/Admin

- Review alerts for pending payments, consultation requests, incomplete customer zones, pause/resume requests, and failed email events.
- Check new subscriptions and payment status before telling a customer the plan is active.
- For Tap manual verification, inspect the exact transaction in Tap first. Confirm it is captured, the reference matches, and amount/currency match. Only then use **Verify captured in Tap**. This action activates the plan and is audited.

- For cash, confirm cash was physically collected before using **Confirm cash received**. This activates the plan and is audited.
- Review package price/duration and current menu before advertising signup.
- Activate demo customers only for presentation/operations previews. Demo records remain marked synthetic and unpaid, and never count as cash or Tap revenue.
- Keep Customer exports and consultation/health information restricted to staff who need it.

Kitchen

- In **Meal Plans**, manage the dish catalogue, ingredient recipe configuration, monthly menu, review, and publication.
- In **Kitchen**, pick the service week and day. Use **Today's production** to focus on the selected day instead of scanning every day's dishes.
- Use dish/meal/portion counts for preparation. Read the parallel allergy, avoid, and preparation notes before cooking.
- Use **Weekly bulk purchasing** to see aggregated ingredient totals. Seeded recipe quantities are estimates until verified by the kitchen; do not place a purchase order from unvalidated estimates.
- Use **Packing** for the client-specific portion/dish detail after bulk ingredients arrive. A-F portion ranges are configurable in Settings and need the client's actual gram ranges entered.

Transport

- Choose the delivery date and review zone/address grouping. Resolve missing or ambiguous zones before route assignment.
- A customer gets one stop with all meals for that day. Friday stops appear only for monthly customers with the Friday add-on.
- Assign one rider to each stop; use the map link to confirm the location. Record delivered/failed status promptly.
- Do not use kitchen screens to resolve customer zones; those belong to Transport/Admin.

Rider

- Sign in through the mobile app after the application is approved.
- Set availability/online state as supported, review assigned stops, use the address/map details, and update delivery status honestly.
- Contact Transport/Admin for an unsafe, inaccessible, or incorrect address. Do not mark a stop delivered until it has been delivered.

Escalation rules

- **Payment says pending after customer paid:** capture transaction reference and amount; check Tap dashboard and payment event log; use manual review only after verifying captured status. Do not tell the customer to purchase again first.
- **No menu appears:** check Meal Plans publication and available-from date; confirm active collection, archived fallback, and production database migrations.
- **Kitchen lists empty:** confirm active paid/demo records, meal selections for the Saturday service week, and valid dish IDs; confirm demo customers were activated for display where necessary. Check recipe config separately for ingredient totals.
- **Rider missing:** confirm rider approval process and active rider record. Current code does not prove that pending applications can be approved in the dashboard.
- **Data/API error:** save the time, role, page, and request/error code; do not attach keys or personal customer data. Escalate to the technical owner.

Kitchen Production Guide

The kitchen's work is built around totals first, customer-specific packing second

Kitchen needs to know what to purchase and prepare for each service day without opening individual customers' records one by one. The operational views separate bulk totals from the details needed to portion and prepare safely.

Monthly menu release

1. Keep the dish catalogue accurate: names, Arabic names where used, active status, calories, ingredients, and recipe quantity/unit per serving.
2. Open **Meal Plans** and start a monthly menu from the catalogue (or import the current PDF if that workflow is available in the deployed build).
3. Review extracted/imported dishes against the source menu. Correct names, meal periods, day, and availability; add or remove dishes as needed.
4. Mark the Kitchen's choice among the available options for each day/meal slot. Customers who miss the weekly deadline use this choice.
5. Review the monthly schedule and publish. The release is expected to be available to customers on Saturday for the service week ahead.
6. Verify the client account sees the published week before relying on it. If no replacement release exists, the backend contains a function to roll a previous release forward, but that job and active collection must be verified on production.

The user interface describes a monthly menu repeating across four service weeks. Confirm the number and dates in the deployed schedule when publishing; monthly date boundaries and Saturday service-week boundaries are not identical.

Customer choice and cutoff

- Customers make meal selections for their package before payment and can manage the next service week in their dashboard.
- Menu options are attached to a service day and meal period. A package limits which meal periods are available; Friday is added only for the eligible monthly Friday add-on.
- Existing Kitchen's-choice defaults are preselected. Customers should not have to reselect unchanged defaults.
- The selection window closes Thursday at 11:59 PM Qatar time. A database job is intended to apply defaults just after the Qatar cutoff. Verify the job in Supabase rather than relying only on the UI message.

Daily production board

Select the service week and the day being cooked. The **Today's production** section should show the selected day's required dishes grouped by dish/meal and portions. Demo records are visually marked **DEMO · UNPAID**; distinguish them from real paid customers in any production handoff.

The safety/preparation section summarizes the details Kitchen must not miss:

- allergies;
- ingredients to avoid or dislikes;
- meal-specific modification or preparation notes;
- selected dish and meal period;

- A-F portion category and count, when configured.

Use these notes to safely plan and portion orders. Allergy flags are not a guarantee that cross-contamination is prevented; the kitchen needs its own food-safety process.

Weekly bulk purchasing

The ingredient-order view aggregates configured recipe quantities over selected active meal records. Conceptually:

`purchase quantity = sum(recipe quantity per serving for each selected serving, after removals/substitutions)`

The output is an ingredient total and the number of meals that use it, not a customer-by-customer purchase list. It can change when menu selections or recipe quantities change.

Important: recipe quantities seeded in the system are estimates. Kitchen must verify each recipe quantity, unit, edible yield/waste assumptions, and portion scaling before treating the computed amount as an authoritative purchase order. A zero/empty total may mean a dish has no recipe configuration, no current selections, inactive/non-matching dishes, or data has not loaded; it does not necessarily mean the kitchen needs zero ingredients.

Ingredient totals in source are based on configured per-serving ingredient quantities and customization logic. Confirm that portion classes A-F correctly affect actual ingredient ordering in the deployed recipe model before relying on different gram sizes for procurement.

Packing and portioning

After bulk items arrive:

1. Open **Packing** for the same service week/day.
2. Use the customer/dish detail to assemble the requested meals.
3. Follow the recorded preparation notes, allergies, avoided ingredients, substitutions, and A-F category.
4. Use the saved A-F min/max gram ranges as guides only after the kitchen confirms them in Settings. Empty ranges are intentionally shown as unset.
5. Reconcile packed meal counts against the production totals before closing the sheet.

Portions and recipes to configure before go-live

- Enter client-approved gram ranges for A, B, C, D, E, and F.
- Review every active dish ingredient list and per-serving quantity; replace seeded estimates with kitchen-approved values.
- Verify units are consistent (g, kg, ml, pieces, etc.) and substitution/removal recipes produce the expected totals.
- Run a controlled sample with two or more customers choosing the same dish at different portion levels and with different notes. Confirm the bulk purchasing total, daily production grouping, safety notes, and individual packing breakdown all make operational sense.
- Sign off the printed/exported production and packing views with the kitchen lead.

Payments and Subscriptions

Source of truth

The production packages records are the active source for package name, price, currency, meal periods, duration, and availability. The web/mobile code includes fallback package examples for resilience; those examples are not proof that current database values match them. Admin should verify packages in the web Subscriptions area before publishing prices.

Known business rules reflected in source/product decisions:

- Monthly subscriptions use 24 service days.
- The one-day and six-day trial offers provide three meals and one snack, subject to current package records.
- The additional 1,600 kcal offer is described as three meals plus two different snack choices at QAR 2,499; verify the active database record before quoting.
- Monthly Friday add-on is QAR 199 for Friday meals/selections and delivery for the month, not per Friday. Trials are excluded.

Do not use this document as a price list when the active database record or customer checkout shows a different current price.

Tap online payments

Intended flow

1. The customer chooses a package, menu, address, and Tap as payment method.
2. A server-side Supabase Edge Function creates the Tap charge using server secrets.
3. Tap returns the customer to `/payment/callback?tap_id=...`; a webhook/callback updates the transaction and plan state.
4. The client app waits for captured status. A captured payment activates the plan; a pending result stays pending.

If the return page loops or says pending

1. Do not make the customer purchase again immediately.
2. Record the customer account email and exact Tap reference/charge ID, amount, currency, and time. Do not copy card data.
3. Check the exact charge in the Tap dashboard and the web **Payments** event log.
4. If the Tap dashboard confirms that exact charge is captured and amount/currency match the transaction, a CEO/Admin may use **Verify captured in Tap**. That is a privileged manual reconciliation action, not a substitute for looking up the charge.
5. If not captured, failed, or mismatched, leave the plan unactivated and resolve through the payment provider.
6. Ask the customer to sign in again and refresh plan status after the review. The auth layer rechecks server-side subscription access when the page regains focus.

Tap checkout and manual reconciliation code exist. This handover review does not certify live merchant credentials, Tap account configuration, deployed webhook delivery, refund/dispute handling, or a successful production end-to-end payment. Perform a low-value authorized live transaction and reconcile/refund it under an approved process before launch.

Cash collection

1. Customer chooses cash and submits the subscription request.

2. The payment transaction is pending; the plan remains inactive.
3. Staff contacts the customer and collects the full amount before the plan begins.
4. Only after receiving the cash, a CEO/Admin opens **Payments** and clicks **Confirm cash received** for that transaction.
5. The database function checks payment method, status, package amount/currency, and authorized staff role; it activates the plan and writes an audit event.
6. Reconcile physical cash against the system report according to the client's bookkeeping process.

Never verify cash because a customer says it was handed over; verify only after the responsible employee confirms actual collection. Do not let Kitchen or Transport approve a cash transaction.

Demo records

Demo customers are synthetic and unpaid. Admin/CEO can activate them for kitchen/routing previews where supported. Activation must not be treated as payment, cash collection, or real revenue. Keep demo records clearly marked in operations and exclude them from financial reports.

Payment provider and notification setup

- Tap is the only online checkout integration evidenced by the reviewed current product source. No Sadad implementation was found.
- Tap account keys and merchant settings are stored server-side through Supabase-managed secrets/vault access; never enter them into client code or this documentation.
- Brevo-backed functions send operational email/reminders; the sender identity, API secret, domain verification, suppression handling, and production cron invocation must be checked.
- Payment settlement, refunds, chargebacks, tax/accounting, and cash reconciliation procedures remain the client's financial responsibility unless a signed services agreement says otherwise.

Technical Handover

System overview

Component	Repository location	Technology / responsibility
Customer and operations website	thk-web/	React, TypeScript, Vite, Supabase JS, Vercel. Public site, customer account, subscription, menu, and role-filtered operations portal.
Mobile application	thk-mobile/	React, TypeScript, Vite, Capacitor 7. Customer and rider app; operations staff are sent to the web portal.
Database and access rules	thk-web/supabase/migrations/ and duplicated migration tree under thk-mobile/	PostgreSQL schema, RLS, views, triggers, RPCs, scheduled jobs. Review migration history before applying changes.
Server-side functions	thk-web/supabase/functions/ and thk-mobile/supabase/functions/	Supabase Edge Functions for payment, webhook, reminders, booking email, health activity reconciliation, and account deletion. The copies are not complete mirrors.
Older prototype	Repository root app.js, index.html, styles.css, JSON translations	Earlier static prototype. It is not the current production app; make current product changes under thk-web/ and thk-mobile/.

Production web URL: <https://trianglehealthykitchen.vercel.app/> Supabase project reference used in source: `teguqlkfmchxucedxvpu` Mobile app ID: `com.trianglehealthykitchen.app`

Main data areas

Names below are the principal data concepts visible in the reviewed source/migrations; this is an operational map, not a complete column-level schema export.

Data area	Purpose and primary users
<code>auth.users</code> , <code>subscribers</code> , <code>staff_members</code>	Identity, customer subscription profile, account state, and staff roles. Auth user creation alone does not mean a paid customer plan is active.
<code>packages</code>	Current package price, currency, duration, meals, availability, and display properties. Active database rows take precedence over code fallback samples.
<code>dishes</code> , <code>ingredients</code> , <code>meal_ingredient_config</code>	Shared menu catalogue, ingredient catalogue, and recipe quantities/units used in purchase totals.
<code>monthly_menu_documents</code> , <code>menu_availability</code> , <code>weekly_menu_selections</code>	Uploaded/reviewed menu source, scheduled dish availability, and each service-week meal choice.
<code>kitchen_production_view</code> , <code>kitchen_ingredient_order_view</code>	Aggregated production/safety and bulk ingredient views for Kitchen; restricted to authorized operations roles.
<code>payment_transactions</code> , <code>payment_logs</code>	Payment request/callback/reconciliation status and audit events. Do not edit production rows manually to bypass guarded RPC workflows.
<code>rider_applications</code> , <code>rider_deliveries</code>	Rider identity/approval and assigned delivery stops/status. The web source reads approved riders for fleet views; pending approval workflow needs confirmation.
<code>notifications</code> , <code>weekly_menu_reminders</code>	Email outcomes and per-customer/service-week reminder tracking.
<code>subscription_pause_requests</code> , <code>consultation/provider booking records</code>	Customer requests and consultation review queues for CEO/Admin.
Health/activity records and BMI report storage	Customer-provided wellness data and optional uploaded health report; treat as sensitive personal data.

Access model

- Staff role values in the app are CEO, Admin, Kitchen, and Transport. Rider is a separate application/approval role. Subscriber is the customer role.
- The web app filters tabs by role and obtains staff role through the `current_staff_role` database RPC.
- Supabase RLS, staff checks, guarded RPC functions, and restricted views are intended to enforce access at the data layer too. Never treat a hidden tab as authorization.
- `transport_subscribers_view` is a narrow transport-specific view; `kitchen_subscribers_view` intentionally exposes only kitchen-relevant active/demo status metadata. Kitchen production and safety views supply meal, portion, allergy, avoid, and prep information without making the full customer directory a kitchen tool.
- Manual payment RPCs are CEO/Admin-only. Review policies/RPC definitions whenever staff roles change.
- Exports may include personal data. Limit export rights and retention to named staff with an operational need.

Data flow

1. Customer signs in and provides subscription, profile, menu, delivery, and payment details.
2. Package, menu, identity, and payment records are validated by the app and server/database constraints.
3. Tap callback/webhook or authorized cash-review RPC updates transaction and plan state.
4. Weekly customer meal choices feed Kitchen's day-specific production totals, ingredient order totals, allergy/preparation notes, and packing details.
5. Delivery address/zone feeds Transport's route list; one rider assignment is created per customer stop/day, not once per meal.
6. Reminder/booking functions write email delivery outcomes for Operations to review.

Configuration names - values are not included here

Web/mobile client build environment

- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY` (public client key; never use the service-role key here)
- `VITE_OPERATIONS_WEB_URL` (optional mobile override; default is the production Vercel domain)

The web has a source fallback for the public Supabase URL/key. Keep deployment values aligned with the intended project. Do not paste any actual keys into handover documents.

Supabase server-side functions and vault

The reviewed functions refer to configuration such as `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `PUBLIC_APP_URL`, Tap configuration (`TAP_SECRET_KEY`, merchant config through protected database helpers), and Brevo configuration (`BREVO_API_KEY`, `SENDER_EMAIL`, provider notification destination). Exact storage path differs between functions: some use Edge Function secrets and some call protected database/Vault helper RPCs. Inspect each deployed function before rotating credentials.

`SUPABASE_SERVICE_ROLE_KEY` and Tap/Brevo secrets must remain server-side.

Native mobile configuration

- Android Firebase uses `thk-mobile/android/app/google-services.json`. The local file is excluded from this documentation bundle and should be supplied from the client-owned Firebase project through a secure channel. Its presence enables Android push setup in the Vite build configuration; absence intentionally skips push registration.
- iOS health sync needs the HealthKit capability, usage description, correct Apple App ID, provisioning profile, and signed build. Store account access is controlled by the client.
- Android health sync uses Health Connect; users with Samsung Health must enable its Health Connect sharing.

- Device location/notifications/health permission prompts must be tested after producing signed builds.

See **External Services Runbook** for step-by-step ownership, setup, safe credential handling, testing, and failure procedures.

Integration inventory and state

Integration	Source evidence	Configuration/acceptance still required
Supabase Auth/Database/Storage	Used by both apps and Edge Functions.	Verify deployed schema/policies, storage access for BMI reports, project ownership, backup plan, and auth email/phone settings.
Vercel	Vite production build and SPA rewrite configuration exist.	Confirm domain, project ownership, environment variables, analytics setting, and production deployment commit. Browser extensions can block Analytics without affecting app behavior.
Tap	Checkout function, callback page, webhook implementation, and manual review exist in duplicated function trees.	Reconcile canonical source, deploy webhook, validate Tap dashboard settings/secret, and complete authorized captured-payment test plus callback/manual-reconciliation cases.
Sadad	No current checkout code located.	Not available until implementation, credentials, callback, security review, and end-to-end verification are separately completed.
Brevo	Booking notification and menu-reminder function code exists in mobile function tree and migration scheduling refers to send-reminders.	Confirm canonical deployed code, sender/domain, vault settings, delivery logs, recipient behavior, and scheduled invocation.
Firebase Cloud Messaging	Android push registration is conditional on local Firebase config.	Configure client-owned Firebase app, Android package, file, permissions, and real-device push test.
Apple Health / Health Connect	Capacitor health plugin integration in mobile source.	Complete platform capability/signing and real-device permission/read/sync tests.
Maps/geolocation	Google Maps URL/coordinate parsing and links, OpenStreetMap Nominatim reverse geocoding, and Qatar GIS FeatureServer zone lookup exist in source.	Test Qatar address lookup, manual entry, map coordinates, external service failure, and zone assignment on real devices; a GPS pin is not guaranteed to fill every address label correctly.
WhatsApp/social/media	wa.me support links, Instagram links, remote Pexels/Unsplash images, and Pravatar demo imagery appear in source.	Confirm official links, contact details, consent-safe message text, and image rights/fallbacks; these are not messaging API integrations.

Critical repository cautions

- 1. Edge Function source is split.** thk-web/supabase/functions/ contains tap-checkout; thk-mobile/supabase/functions/ contains additional functions including tap-webhook, send-reminders, send-booking-notification, reconcile-activity, and process-account-deletion. Do not deploy one folder blindly. Confirm which code is deployed to the production Supabase project, diff the duplicates, choose a canonical source, and bring the other tree into agreement.
- 2. Migration history may be incomplete.** The repository README records missing-local migration history in the linked Supabase project. Do not run a bulk supabase db push until the remote/local history is reconciled. Review each migration and apply through the client-approved release procedure.
- 3. Database migrations are duplicated.** Compare web and mobile migration trees before database changes; apply each intended migration once to the correct Supabase project.
- 4. Current repository contains local build artifacts and configuration outside this documentation set.** Do not distribute node_modules, dist, www, .temp, .env*, google-services.json, signing files, or local IDE/AI state as handover material.

5. **Demo SQL scripts are destructive or mutate demo records.** Read their comments and target filters; do not run against live customers.

Useful source entry points

- Web routing and customer pages: `thk-web/src/App.tsx`, `src/pages/`, `src/components/`
- Staff access: `thk-web/src/lib/auth.tsx`
- Operations workspaces and role modules: `thk-web/src/pages/OperationsWorkspace.tsx`
- Meal signup and payments: `thk-web/src/components/SubscriptionFlow.tsx`,
`src/pages/PaymentCallbackPage.tsx`
- Kitchen data transforms/views: `thk-web/supabase/migrations/`
- Mobile role routing and native integration: `thk-mobile/src/App.tsx`, `src/lib/health.ts`, `capacitor.config.ts`

Deployment and Maintenance

Account ownership

The client should own the production Vercel project/domain, Supabase project, payment account, email sender, Firebase project, Apple Developer account, Google Play Console account, and any signing credentials. Invite the developer as a named user with only the role needed. Keep a separate secure inventory of owners, billing contacts, recovery contacts, MFA, and renewal dates; do not put passwords or recovery codes in this manual.

Web development and release

The current website lives under `thk-web/` and requires Node.js 24 per its package metadata.

```
``sh cd thk-web npm install npm run lint npm run build npm run preview ``
```

Before a production release:

1. Review the code change and database impact.
2. Confirm required environment variables in the correct Vercel environment (preview and production are separate).
3. Review and apply any database migration through the agreed release process. Back up first; check remote migration history because the local README reports historical drift.
4. Deploy a Vercel preview and run the relevant customer/role workflow checks.
5. Promote/deploy the approved commit to production using the client-owned Vercel workflow.
6. Verify the deployed commit, login, customer signup, menu, payment return, operations roles, and mobile-web entry point.
7. Record deployment date, commit, migration IDs, checks, owner, and rollback target.

To roll back a web-only defect, use Vercel's previous known-good deployment. A Vercel rollback does not reverse database changes; use a reviewed forward fix or a verified database restore plan.

Mobile build and release

The mobile application is under `thk-mobile/`. Android and iOS share the customer UI, but native signing, permissions, Firebase, HealthKit/Health Connect, and store policies are platform-specific.

```
``sh cd thk-mobile npm install npm run typecheck npm run lint npm run build npx cap sync ``
```

- Android emulator/debug APK: `npm run build:apk` (requires Android SDK/Gradle environment).
- Android store bundle: use Android Studio **Build** → **Generate Signed Bundle / APK**, with the client-owned Play Console app and protected release keystore.
- iOS assets sync: `npm run build:ios`; open `ios/App/App.xcworkspace` in Xcode, set the client Team/signing profile/capabilities, archive, and upload using the client-owned Apple account.
- Increase the platform build/version number for each store update. Preserve and back up the same Android signing key; losing it can block updates.
- Test signed release builds on physical iPhone and Android devices. Emulator preview is not store acceptance.
- Store listing, privacy disclosures, age/content ratings, screenshots, review, and account fees are client-owned dependencies unless a separate written delivery includes them. Store publication has not been established by this handover.

Supabase/database/functions release

1. Identify the canonical migration and Edge Function tree first. The repository currently duplicates migrations and splits functions between web and mobile folders.

2. Compare local files with the deployed production function versions and deployed migration history.
3. Back up or confirm point-in-time recovery is available; rehearse a restore to a non-production environment.
4. Review policies, function grants, triggers, cron jobs, and secrets for every change.
5. Apply one reviewed migration at a time through the client-approved procedure. Do not bulk push while migration history is unresolved.
6. Deploy the intended functions and inspect function logs.
7. Confirm send-reminders-job, kitchen-choice default processing, account data cleanup schedule, and any required network extension jobs in the production project.
8. Verify exact post-release role access and audit records.

Never copy production database data into preview/local without sanitization. Do not manually update payment/subscriber tables to bypass RPC validation and audit history.

Backups and recovery

- Define a client-owned backup schedule and retention period based on the Supabase plan and the business's legal/accounting needs.
- Confirm database backup/PITR capability and storage backup coverage; database backups may not include all Storage objects or secrets.
- Store export/encryption keys separately from backups.
- Test restoration into a separate Supabase project before calling a backup usable.
- Record recovery time objective (RTO), recovery point objective (RPO), incident owner, and client notification path. These have not been contractually set in the reviewed agreements.

Routine maintenance calendar

Frequency	Owner	Work
Each menu cycle	Admin/Kitchen	Verify next week's menu release, kitchen choice, package eligibility, recipe completeness, ingredient estimates, and deadline jobs.
Daily operations	Admin/Transport/Kitchen	Review payment exceptions, missing zones, delivery assignments, failed email events, and today's production/packing totals.
Monthly	System owner	Review staff access, provider charges/renewals, Vercel/Supabase errors and usage, backups, failed jobs, and domain/email health.
Each release	Developer/system owner	Review code, migration, security policies, preview, acceptance scenarios, release notes, and rollback path.
Quarterly or after incident	Client + technical owner	Rehearse restore, rotate access where needed, review data retention, and confirm vendor/account owners.

Support and maintenance terms

No response-time SLA, included support period, maintenance price, hosting responsibility, or uptime commitment should be implied by this document. Agree those items in writing. A practical maintenance agreement should define supported hours/time zone, severity levels, response targets, included defect fixes, paid enhancement work, backups/recovery ownership, third-party account costs, security updates, and termination/handover steps.

Launch Checklist and Known Limits

Use this checklist with the client's named business, kitchen, transport, and technical owners. Mark each item **Pass**, **Fail**, or **Not tested** and attach a safe reference (no passwords, card data, or client health records).

A. Ownership and access

- [] Client owns Vercel/domain and can deploy or roll back.
- [] Client owns Supabase and can manage users, billing, backups, and secrets.
- [] Client owns Tap, Brevo, Firebase, Apple Developer, and Google Play accounts as applicable.
- [] Staff have individual accounts and correct CEO/Admin/Kitchen/Transport roles.
- [] Role access checked in both UI and database (Kitchen cannot access customer/transport-only information; Transport cannot approve payments).
- [] Rider application approval has a confirmed supported workflow. **Current operations source only loads approved riders; pending approval control was not found.**
- [] Recovery contacts and MFA set; departing staff can be removed without sharing/resetting other people's credentials.

B. Customer signup and account

- [] English and Arabic landing/navigation, forms, errors, legal links, and account views reviewed on mobile and desktop.
- [] Email signup/login, password reset from the live domain, and session recovery tested.
- [] Phone country-code selection and OTP delivery channel tested using production provider settings; verify whether WhatsApp OTP is actually configured before promising it.
- [] Profile fields reject invalid/zero weight or height and explain optional BMI report clearly.
- [] Existing customer can sign in, renew, see their plan, and request deletion without being forced into a duplicate purchase.
- [] Account deletion request and 30-day inactive-customer cleanup are tested; published policy and actual cron schedule match.

C. Plan, menu, and kitchen

- [] Active package prices, currency, service days, meal periods, Friday eligibility, and meal limits match business-approved values.
- [] One-day and six-day trials show 3 meals + 1 snack if confirmed by the business.
- [] 1,600 kcal, 3-meal/2-different-snack package is active at the approved price (currently requested as QAR 2,499).
- [] Friday add-on is QAR 199 per monthly subscription and includes Friday meals/selections and one Friday delivery stop per service Friday.
- [] Admin menu import/upload, editable review, additions/deletions, publication, Saturday customer availability, and future week selection tested.
- [] Previous release fallback fills a missing week and does not duplicate menu-availability rows.
- [] Customers can keep preselected Kitchen choice, edit desired meals, and cannot exceed package meal periods.
- [] Weekly cutoff is Thursday 11:59 PM Qatar time; job applies Kitchen's choice only after cutoff; reminders arrive approximately 24 hours beforehand.

- [] Kitchen sees today's required dish/meal totals and relevant allergy/preparation notes; customer addresses/zones remain with Transport/Admin.
- [] Purchase quantities use kitchen-verified recipe amounts/units, include substitutions/removals correctly, and handle portion A-F scaling as intended.
- [] Packing shows client-specific dish and portion details needed after bulk purchase.
- [] Friday delivery list includes only monthly add-on customers; delivery is one stop per customer/day with all meals together.
- [] Test at least two customers with different dishes, shared dishes, different A-F portions, allergy notes, customizations, and different zones.

D. Payments

- [] Tap production keys and account settings verified in Tap and stored only in server-side configuration.
- [] Tap redirect URL is the live domain callback and Tap webhook points to the correct production Edge Function.
- [] A permitted live test produces the correct transaction, currency, package/Friday total, callback, webhook, and plan activation.
- [] Delayed callback case stays pending and is reconciled without asking the customer to repurchase.
- [] CEO/Admin manual Tap verification works only after a human confirms the exact captured charge, currency, and amount.
- [] Cash stays pending until physically collected; CEO/Admin verification activates it and creates an audit event.
- [] Failure, duplicate callback, cancellation, refund, and chargeback procedures are agreed. Refund/chargeback UI/process is not established by this documentation review.
- [] Sadad is not described as available unless separately implemented and acceptance-tested.

E. Email, mobile, and integrations

- [] Brevo API configuration, sender/domain verification, booking notifications, menu reminder delivery, and failure logs tested.
- [] Supabase cron jobs verified in production: weekly default, send-reminders, and 30-day cleanup.
- [] Canonical Supabase Edge Function tree chosen and deployed; tap-webhook and other mobile-tree functions are not assumed to exist just because files are in the repo.
- [] Android Firebase config belongs to client's project, is secure, and push is tested on a real device.
- [] Apple Developer team, app identifiers, HealthKit capability, privacy usage text, and signed iOS build are configured.
- [] Apple Health and Android Health Connect consent/read/sync are tested on real devices; Samsung Health sharing steps documented for users.
- [] Address autocomplete/locate/manual entry and Qatar zone mapping verified with real addresses; map marker accuracy checked.
- [] Store privacy, data safety, account deletion, nutrition/health, payment, and location disclosures match actual collection/use.

F. Release and operations acceptance

- [] Web build, role sign-in, customer signup, menu, payment, kitchen, route, Arabic, and mobile responsive checks pass.
- [] Android signed build installed/tested on physical phone and release signing key backed up.

- [] iOS signed build installed/tested on physical iPhone. Store publication is separately scheduled with client-owned account.
- [] Demo customers are labeled unpaid and excluded from sales totals.
- [] Customer, payment, kitchen, packing, and route CSV exports open and contain expected columns.
- [] Backup restore is demonstrated outside production.
- [] Operations and client-owner training completed; support owner and escalation contact named.
- [] Client accepts the agreed scope and signs the aligned commercial/license documents.

Known limitations and handover conditions found in repository review

1. **Payment provider:** Tap implementation is present but live transaction/callback behavior needs client-owned credential and webhook verification. Sadad was not found in current source.
2. **Supabase functions are split:** checkout appears under the web tree; webhook, reminders, booking email, health reconciliation, and account deletion appear under the mobile tree. Production deployment must be reconciled with this layout.
3. **Migration history:** root README warns that the linked production project has local/remote migration-history drift. Bulk database push is not a safe assumed handover step.
4. **Menu/kitchen data quality:** recipe ingredient quantities are seeded estimates and require Kitchen approval. Portion ranges A-F require real gram values. Procurement should not rely on defaults until confirmed.
5. **Rider approval:** the reviewed operations dashboard shows approved riders but no pending application approval action. Confirm a usable approval process before onboarding riders.
6. **Mobile publication:** source projects exist but the reviewed state does not establish signed, store-approved iOS/Android releases. Client store memberships and signing are prerequisites.
7. **Push notifications:** Android registration depends on Firebase configuration; the Firebase config file is deliberately excluded from this package.
8. **Arabic completeness:** translation infrastructure and many Arabic strings exist. A complete screen-by-screen Arabic review is still an acceptance task; this package does not certify every string.
9. **Operational reminder scheduling:** SQL/function code exists; actual production cron, secrets, and delivery require verification.
10. **Support/warranty:** response times, uptime, included maintenance period, and ongoing fees are not established here. Agree them separately in the signed commercial terms.

Sign-off

Area	Result	Owner	Date / evidence reference
Customer signup and plans			
Menu and Kitchen			
Payments			
Transport and riders			
Arabic and mobile			
Deployment, backups, and recovery			
Client acceptance			

External Services Runbook

This runbook identifies services outside the application code, what THK uses them for, who should own them, how to configure or verify them, and what to do when they fail. Create provider accounts in the client's name. Keep all credentials in the provider or its approved secret store; this runbook intentionally contains no credential values.

Service inventory

Service/source	Use in THK	Key operational point
Supabase	Authentication, PostgreSQL, Storage, RLS, Edge Functions, cron jobs	System of record. Preserve migration history and verify security policies/functions before release.
Vercel	Production website hosting and deployment	Deploys the web app; keep the Supabase project and domain settings aligned.
Brevo	Supabase Auth SMTP and transactional booking/menu emails	SMTP key and API key are different credentials and are consumed through different paths.
Tap Payments	Online checkout, charge status, webhook/callback	Reconcile exact captured charges; never trust the browser return page alone.
Firebase Cloud Messaging	Android push registration	Android push is skipped unless <code>google-services.json</code> exists at the expected native path.
Apple HealthKit	Optional iPhone health-data read/sync	Requires client Apple Developer membership, HealthKit entitlement, signing, and user consent.
Android Health Connect	Optional Android health-data read/sync	Samsung Health must share data with Health Connect first.
Qatar GIS zones service	Converts a map point into Qatar ZONE_NO	Source code uses the Qatar GIS public FeatureServer query; if unavailable, zone stays unset and Transport must review.
OpenStreetMap Nominatim	Reverse-geocodes GPS coordinates into address labels	Current source calls the public reverse endpoint without an API key. Treat the result as a suggestion, provide manual entry, and check current usage terms/provider suitability before higher-volume production use.
Google Maps	Opens/parses customer/rider map links and coordinates	Current source builds links and parses coordinates; it does not show a Google Maps API key or Places autocomplete integration.
WhatsApp (wa.me)	Opens a prefilled support or rider-contact conversation	This is an outbound deep link, not evidence of WhatsApp Business API or WhatsApp OTP delivery. Check current number and privacy-safe message content.
Pexels / Unsplash	Food/marketing fallback images	Remote image availability can affect presentation. Replace with licensed, client-approved images for a production brand.
Pravatar	Community demo avatar images	Synthetic/community presentation use only; do not represent generated avatars as real named customers.
Instagram	Footer/social navigation	Confirm one correct official handle on web and mobile before launch; the source contains different web/mobile handle strings.

1. Supabase

Ownership and initial access

1. Keep the production project under the client's organization, with at least two client-controlled owner/recovery contacts and MFA.

2. Invite developers as named team members with only the access needed. Do not share the owner login.
3. Keep the project URL and anon/public key in the client app configuration. Never use the service-role key in web/mobile code.
4. Store server keys only in Supabase Edge Function secrets or the protected database Vault/RPC mechanism expected by the function. The service-role key bypasses ordinary row policies and must never be sent to a browser/device.

Auth and email setup

1. Set the production Site URL and redirect allow-list to the deployed domain and supported auth/payment return paths.
2. Test signup confirmation, sign-in, password recovery, session expiry, and callback handling on the live HTTPS domain. A recovery link must return to a valid app session and route.
3. If using custom SMTP for Supabase Auth, use Brevo's **SMTP relay hostname** (`smtp-relay.brevo.com`) and the SMTP port supported by the account (the configured screen previously showed 587). Use Brevo's SMTP username and a generated SMTP key in Supabase; this password is **not** the Brevo API key and is not the account login password.
4. Confirm sender/domain authentication and that Supabase Auth confirmation/reset messages arrive independently from THK booking/menu transactional messages.
5. Phone/WhatsApp OTP delivery depends on Auth provider setup. A WhatsApp support link in the app does not configure OTP delivery.

Database, storage, functions, and scheduled jobs

1. Reconcile local and remote migration history before applying migrations. The repository records historical drift; do not use an unreviewed bulk push.
2. Review RLS and RPC permissions after each migration. Check as anonymous, customer, Kitchen, Transport, Admin, and CEO accounts.
3. Confirm the `bmi-reports` Storage bucket and policies restrict each uploaded report to the intended authorized flow. Treat reports as sensitive health data.
4. Reconcile the split Edge Function trees in the repository before deployment (see Technical Handover). Check production function source/deployed versions rather than assuming a local copy is live.
5. In Supabase logs, verify function invocations/errors and the cron jobs for menu defaults, menu reminders, and expired-customer cleanup. Confirm they target this production project and use the correct Qatar-week logic.
6. Confirm backup/PITR availability, storage coverage, and a restore drill. A database backup may not restore object files, function secrets, or external provider configuration.

If something fails

- **Auth redirect/reset failure:** check Auth Site URL/redirect allow-list, SMTP logs, and whether the recovery link is stale; generate a fresh recovery link.
- **HTTP 401/403:** verify the current user/session, role assignment, RLS policy, and RPC grants. Do not solve by exposing service-role credentials.
- **HTTP 400 on a table/RPC:** read the Supabase error body and compare the deployed schema/function with local migration history before changing data.
- **Cron does not run:** inspect `cron.job` and `job-run/network` logs, function endpoint, secrets, and `pg_net/extension` configuration.

2. Vercel

1. Keep the Vercel project under the client's account or organization and connect the intended GitHub repository/production branch.

2. Set the project root to `thk-web/` if that is not already the configured Vercel root. Confirm the build command is `npm run build`, output directory is `dist`, and SPA routes rewrite to `index.html`.
3. Configure `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` in Preview and Production. Set `VITE_OPERATIONS_WEB_URL` in the mobile build only if the default production URL should be overridden.
4. Use a Preview deployment to verify each release before promoting to Production. Confirm `/login`, `/account`, `/payment/callback`, `/dashboard`, and role-specific routes load directly.
5. After a production deploy, record the Git commit and deployment URL; exercise sign-in, meal selection, and payment-return routing.
6. Vercel Analytics can be blocked by browser privacy extensions (`ERR_BLOCKED_BY_CLIENT`). Distinguish that client-side block from application/API failures; enable analytics for the Vercel project only if the owner wants it.
7. Roll back a web-only regression to a prior Vercel deployment. Database changes need a separate reviewed recovery/fix; a frontend rollback does not reverse SQL.

3. Brevo

THK uses Brevo through **two distinct channels**. Configure and test both:

Supabase Auth transactional SMTP

- Set a verified sender/domain in Brevo and finish the DNS authentication steps Brevo requests.
- In Supabase Auth email/SMTP settings, enter Brevo's SMTP relay, the account's SMTP username, and a generated SMTP key.
- Use the SMTP key, not the Brevo dashboard password and not the API key.
- Test a new-user confirmation and a password reset on the production domain. Check spam placement, From name/address, reply handling, and rate limits.

THK Edge Function emails

- The booking-notification function source reads `BREVO_API_KEY` and `SENDER_EMAIL` from Edge Function environment secrets.
- The menu-reminder function source calls the protected `get_brevo_config()` database helper, which reads a `key/sender` from the project's Vault configuration.
- Because the two source copies use different configuration paths, first reconcile the canonical production function source. Put the API key only in the exact server-side secret/Vault location used by that deployed function. Do not paste it into `VITE_*`, frontend code, GitHub, or this manual.
- Confirm `send-booking-notification` sends the expected customer/operations booking messages and writes success/failure logs.
- Confirm `send-reminders` runs from the scheduled job, selects only customers who still need to choose meals, sends before the Thursday Qatar-time cutoff, and records the `(customer, service week)` reminder so a repeated invocation does not spam them.
- Check Brevo transactional email logs and the THK Notifications view for accepted, bounced, blocked, or failed outcomes. A 2xx provider response means accepted for processing, not guaranteed inbox placement.
- Rotate a compromised API key in Brevo and in the correct Supabase secret store, redeploy/restart dependent functions if needed, then send a controlled test.

4. Tap Payments

1. The client owns the Tap merchant account and activates live processing with Tap. Confirm the environment (`test/live`), enabled payment methods, currency, settlement, and account status directly in Tap.
2. Store the Tap secret only through the protected Supabase configuration path used by `get_tap_config()`; confirm the deployed checkout function can retrieve it. Do not place the secret in Vercel `VITE_*`, app source, or a mobile

bundle.

3. Set the redirect URL to `https://trianglehealthykitchen.vercel.app/payment/callback` (or the approved active domain).
4. Set the webhook/post URL to the correct production Supabase tap-webhook Edge Function. Verify the URL and signing/HMAC behavior against the deployed function. The current checked-in webhook copy is under `thk-mobile/supabase/functions/`; reconcile it with the deployed function before editing/deploying.
5. Make one authorized end-to-end payment using an approved test/low-value method. Confirm the transaction row, Tap dashboard charge, webhook/callback logs, exact amount/currency, and active customer plan.
6. Simulate delayed callback behavior without creating a second charge. CEO/Admin manually reviews only after confirming the exact charge is captured in Tap.
7. Agree who handles refunds, disputes, chargebacks, settlement reconciliation, and customer receipts. Those procedures are not fully represented by the current app screens.
8. Sadad is not in the reviewed app code. Do not add Sadad credentials or advertise it as integrated until a separate implementation is built and tested.

5. Firebase Cloud Messaging (Android)

1. Create/choose the client-owned Firebase project and register the Android app with package/application ID `com.trianglehealthykitchen.app`.
2. Download the Android client configuration file from Firebase and put it at `thk-mobile/android/app/google-services.json`. Keep it out of public Git and out of this documentation archive.
3. Confirm Gradle's Google Services/Firebase Messaging setup and the application ID match Firebase. Rebuild and sync the Capacitor project after adding configuration.
4. Sign in on a physical Android device, grant notification permission, confirm FCM token registration, then send a controlled test message through the intended server notification path.
5. If there is no Firebase configuration, current app code intentionally skips Android push registration. Email reminders continue as a separate channel; do not claim Android push is live until tested.
6. Store Firebase Admin/service credentials, if later needed by server push, only in a protected server secret store. The Android client JSON is not a server admin key, but it still belongs to the client's project and should not be published casually.

6. Apple Health and Android Health Connect

iPhone / Apple Health

- The app asks for customer consent to read steps, distance, calories, and weight. It does not sync until the customer initiates it.
- The iOS project has a HealthKit entitlement and usage description. The client still needs an active Apple Developer membership, the matching App ID capability, provisioning/signing, and a signed device build.
- The client has not yet paid for the Apple Developer subscription. Therefore iOS HealthKit store/distribution readiness is blocked until that account is active and signing is configured. A simulator/source check is not a production-device validation.
- Test grant, deny, revoke, empty data, and successful sync on an iPhone. Confirm the privacy policy accurately describes health data.

Android / Health Connect

- The app queries Health Connect through the Capacitor health plugin after the user grants access.

- On Samsung phones, the user must enable Samsung Health sharing into Health Connect first. Test on a physical Samsung/Android device as well as a generic Android device.
- Test unavailable Health Connect, denied permissions, no samples, and successful daily steps/distance synchronization.

Health integrations are optional wellness features; do not present their output as diagnosis or medical advice.

7. Location and maps

- The app can use device geolocation, parse Google Maps URLs/coordinates, and create outgoing Google Maps links. The reviewed source does not use a Google Maps API key or provide Places autocomplete.
- The `Locate` flow calls OpenStreetMap Nominatim reverse geocoding to suggest address labels, and Qatar's GIS FeatureServer to resolve a `ZONE_NO` from the GPS point. These are external network services; they can fail, time out, or return incomplete address data.
- If either lookup fails, the customer must be able to enter/review the delivery address manually. Treat zone as unconfirmed if the lookup is blank; Transport should check it before assigning a rider.
- Verify coordinates inside Qatar, edge-of-zone addresses, a manually entered home/office/gym address, denied location permission, and network failure. Ensure the location pin and human-readable address refer to the same place.
- Before increasing traffic, check current Nominatim and Qatar GIS terms/availability and decide whether the business needs a supported geocoding service or server-side proxy. Do not assume a public endpoint has a commercial uptime commitment.

8. WhatsApp, social links, and remote images

- WhatsApp buttons open `wa.me` chats; this is not a WhatsApp Business API integration. Confirm the business number, country-code formatting, prefilled text, and rider/client support routing.
- Do not place passwords, payment-card data, health measurements, BMI reports, or detailed allergy information in WhatsApp URLs/messages. The current client-support link can include the customer's name in the prefilled text; review consent and minimize data in these message templates.
- Confirm website and mobile Instagram links point to the same official account. The reviewed source has different handle strings between the two clients.
- Pexels/Unsplash supply remote image URLs; Pravatar supplies generated community avatars. Ensure the owner has appropriate usage rights/approval and replace demo-looking assets before client sign-off. Remote images require network access; test their fallback/loading behavior.

Provider incident checklist

For any external service failure, record: provider, affected environment, time in Qatar, affected workflow, safe transaction/request reference, error code, recent deployment or key rotation, and whether a fallback worked. Do not attach tokens, keys, password-reset links, full payment details, medical reports, or screenshots with customer PII. Assign one owner to contact the vendor and another to keep customer operations moving using the documented fallback.